zeroriscy_alu

OVERVIEW

The Arithmetic Logic Unit (ALU) is one of the most critical components of the processor, responsible for executing all arithmetic, logical, shift, and comparison operations. It receives operands from the execution stage and generates results required for computation, branching decisions, and control flow management.

This module plays a central role in instruction execution by performing operations such as addition, subtraction, logical manipulation, shifting, and condition checking.

The ALU is designed with hardware efficiency in mind and is optimized using:

- A shared adder for multiple operations
- Efficient bit manipulation techniques for shifting
- Unified comparison logic using subtraction-based evaluation

These optimizations reduce hardware complexity while maintaining high performance and reliability.

1. ARITHMETIC OPERATIONS

The ALU performs fundamental arithmetic computations required during instruction execution, including:

- Addition (ADD)
- Subtraction (SUB)

These operations are widely used in address calculation, arithmetic instructions, and control operations.

2. LOGICAL OPERATIONS

The module supports standard bitwise logical operations:

- AND
- OR
- XOR

These operations are commonly used in masking, bit manipulation, and logical decision-making.

3. SHIFT OPERATIONS

The ALU performs three types of shift operations:

- Logical Left Shift (SLL)
- Logical Right Shift (SRL)
- Arithmetic Right Shift (SRA)

Logical shifts move bits without considering sign, while arithmetic right shift preserves the sign bit for signed number operations.

4. COMPARISON OPERATIONS

The ALU supports various comparison instructions used for branching and decision-making. These include:

- Equal / Not Equal
- Greater Than / Less Than
- Signed and Unsigned comparisons

- Set-Less-Than variants

These comparisons are essential for branch instructions and conditional execution.

5. MULT/DIV SUPPORT

The ALU also supports integration with the multiplier/divider unit by sharing the adder logic.
This improves hardware utilization and avoids unnecessary duplication of arithmetic resources.

KEY FEATURES

- Shared adder design for reduced hardware usage
- Support for both signed and unsigned comparisons
- Efficient bit-reversal technique for left shift operations
- Set-less-than operation handling
- Extended 34-bit adder output for carry and sign support
- Seamless integration with multiplier/divider unit

INPUT AND OUTPUT SIGNALS

Inputs / Control and Operand Inputs:

- operator_i [ALU_OP_WIDTH-1:0] -> Operation selector provided by the instruction decoder
- operand_a_i [31:0] -> First operand input
- operand_b_i [31:0] -> Second operand input

Mult/Div Inputs:

- multdiv_operand_a_i [32:0] -> Operand A for multiplication/division operations
- multdiv_operand_b_i [32:0] -> Operand B for multiplication/division operations
- multdiv_en_i -> Enables multiplier/divider mode. When enabled, the ALU uses mult/div operands instead of normal execution operands.

Outputs:

- result_o [31:0] -> Final ALU output result
- adder_result_o [31:0] -> Standard adder output
- adder_result_ext_o [33:0] -> Extended adder output used for carry and sign handling
- comparison_result_o -> Final comparison result
- is_equal_result_o -> Equality flag indicating whether Operand A equals Operand B

OPERATION BREAKDOWN: SHARED ADDER LOGIC

The ALU uses a single shared adder for multiple operations, including:

- Addition
- Subtraction
- All comparison operations

This significantly reduces hardware area and improves design efficiency.
Subtraction is implemented using two's complement arithmetic:

A - B = A + (~B + 1)

This operation is controlled using: adder_op_b_negate
By reusing the same adder for both arithmetic and comparison logic, the design becomes simpler and more

power-efficient.

MULT/DIV MODE

When multdiv_en_i = 1, the ALU switches from normal execution operands to:

- multdiv_operand_a_i
- multdiv_operand_b_i

Otherwise, standard operands (operand_a_i, operand_b_i) are used.

This allows smooth integration with multiplication and division operations.

SHIFT OPERATIONS (DETAILED)

Left Shift (SLL):

The left shift operation is implemented using an efficient bit-reversal technique:

1. Reverse the input bits
2. Perform a right shift
3. Reverse the result back

This approach reduces the need for separate left-shift hardware.

Right Shift:

Two right-shift operations are supported:

- Logical Right Shift (SRL)
- Arithmetic Right Shift (SRA)

Arithmetic right shift preserves the sign bit, making it suitable for signed operations.

COMPARISON LOGIC

Comparison operations are performed using subtraction-based evaluation.

Step 1: Subtraction

The ALU computes: A - B

Step 2: Equality Check

is_equal = (A - B == 0)

This determines whether both operands are equal.

Step 3: Greater/Less Decision

The ALU uses sign bits, result MSB, and signed/unsigned operation mode to determine whether one operand is greater than or less than the other.

This unified comparison logic avoids dedicated comparator hardware and improves efficiency.

SUPPORTED COMPARISONS

- EQ : Equal
- NE : Not Equal
- GTU : Greater Than (Unsigned)
- GEU : Greater Than or Equal (Unsigned)
- LTU : Less Than (Unsigned)
- LEU : Less Than or Equal (Unsigned)
- GTS : Greater Than (Signed)

- GES : Greater Than or Equal (Signed)
- LTS : Less Than (Signed)
- LES : Less Than or Equal (Signed)

RESULT GENERATION

The final output is selected based on the value of operator_i.
Result Selection:

- Arithmetic operations -> adder_result
- Logical operations -> Bitwise operation result
- Shift operations -> shift_result
- Comparison operations -> {31'b0, cmp_result}

This ensures that the ALU produces the correct output for every instruction type.

INTERNAL DESIGN TECHNIQUES

Bit Reversal Trick:
This technique is used for efficient implementation of left-shift operations. Process: Reverse bits, perform right shift, reverse bits again. This avoids designing separate left-shift hardware and improves resource efficiency.
Extended Adder (34-bit):
The ALU uses a 34-bit extended adder to handle carry generation, overflow detection, and sign management. This is especially important for multiplication/division support and signed comparisons.
Unified Comparison Logic:
Instead of separate comparator circuits, all comparison operations are derived from a single subtraction operation, sign-bit analysis, and equality checking. This improves hardware reuse and reduces overall circuit complexity.

SUMMARY

The zeroriscy_alu is a highly optimized combinational execution unit that performs all major arithmetic, logical, shifting, and comparison operations required by the processor.
It:

- Executes core arithmetic and logical instructions
- Uses a shared adder to reduce hardware usage
- Supports efficient signed and unsigned comparisons
- Implements low-cost shift operations using bit manipulation techniques
- Integrates smoothly with multiplier/divider units

Its efficient architecture makes it a critical component of the Zero-Riscy processor, ensuring both high performance and optimized hardware utilization.